Django REST Framework

Simone Ricci

RESTful API

RESTful API

An **API** (Application Programming Interface) is a shorthand way to **describe how two computers communicate** directly with one another.

An API acts as an **intermediary layer** that processes data transfers between systems, letting **open their application data and functionality** to external third-party developers, business partners, or companies.

For web APIs, which exist on the world wide web, the dominant architectural pattern is know as **REST** (**RE**presentational **S**tate **T**ransfer)

First introduced in 2000 by computer scientist Roy Fielding in his doctoral thesis.

REST provides developers with a relatively high level of **flexibility** and **freedom**.

This flexibility is just one of the reasons why the REST API is the most popular method for connecting components and applications in a **microservice architecture.**

Key aspects of RESTful APIs

1) Resources: In a RESTful API, resources are the **fundamental entities** that are exposed and manipulated. A resource **can be any piece of data**, such as a user, a book, or an order.

2) Uniform Interface: RESTful APIs adhere to a uniform interface, which means they use a **consistent set of HTTP methods** (**GET, POST, PUT, DELETE**) to perform operations on resources. Each HTTP method has a specific purpose:

- GET retrieves a resource
- POST creates a new resource
- PUT updates an existing resource
- DELETE removes a resource

3) Stateless: each request from a client to a server must contain all the necessary information for the server to understand and process the request.
The server does **not maintain any session or state** information about the client between requests. This simplifies scalability and allows for better fault tolerance.

3) Representation: Resources are represented in a specific format, such as **JSON** (JavaScript Object Notation) or **XML** (eXtensible Markup Language). Clients can request different representations of a resource based on their needs.

RESTful APIs in modern web development

Interoperability: RESTful APIs provide a **standardized way** for different systems and technologies to communicate with each other.

Client-Server Separation: RESTful APIs enforce a clear separation between the client and server, enabling **independent development** and scalability. It allows flexibility and **modularity** in building complex applications.

Scalability: Stateless communication means that each request can be handled independently, making it easier to **distribute the load** across multiple servers or scale horizontally.

Mobile and Web Applications: RESTful APIs are widely used in building mobile and web applications. Thanks to interoperability.

Integration and Third-Party Development: By exposing resources and operations, APIs allow other developers and services to leverage the functionalities of an application or platform.

Future-Proofing: RESTful APIs are **technology-agnostic** and can be consumed by various clients, ranging from web browsers to mobile apps and IoT devices

How REST APIs work

The Endpoint: /api/books

GET /api/books - Retrieve a list of all books.

GET Response

```
HTTP 200 OK

[
  {
    "id": 1,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "published_date": "1925-04-10"
  },
  {
    "id": 2,
    "title": "To Kill a Mockingbird",
    "author": "Harper Lee",
    "published_date": "1960-07-11"
  }
]
```

POST /api/books - Create a new book.

```
POST /api/books
Content-Type: application/json

{
  "title": "1984",
  "author": "George Orwell",
  "published_date": "1949-06-08"
}
```

POST Request

```
HTTP 201 Created

{
  "id": 3,
  "title": "1984",
  "author": "George Orwell",
  "published_date": "1949-06-08"
}
```

POST Response

Alternative API architectures

GraphQL: GraphQL is an open-source query language and runtime that allows clients to request specific data requirements from the server (enables clients to specify the exact data they need)

gRPC: gRPC (Google Remote Procedure Call) is a high-performance, language-agnostic framework for building distributed systems.

SOAP: SOAP (Simple Object Access Protocol) is an **XML-based** messaging protocol used for exchanging structured information between networked applications. SOAP **defines a set of rules** for formatting messages, including request and response envelopes, headers, and body.

JSON-RPC: it is a remote procedure call (RPC) protocol that uses JSON (JavaScript Object Notation) as the data format.

Falcor: Falcor is a JavaScript library developed by Netflix that implements a data-fetching pattern known as "JSON Graph." It allows clients to request and manipulate specific parts of a JSON-like graph directly from the server.

GraphQL

It is query language for APIs and a server-side runtime for executing those queries. Allows clients to request exactly the data shape they need via a **single endpoint**.

1) Core Concepts:

   Schema & Types: Defines available data types and relationships in a **strongly-typed schema**.

   Single Endpoint: One URL (e.g., /graphql) that accepts all queries.

   Query Shape: Clients send a **field-based selection set**; server returns JSON matching that shape.

2) Why Use GraphQL?

   No over- or **under-fetching**—only fields you ask for.

   Automatic schema documentation; IDE auto-completion

   Single Round-Trip: **Nest related data in one request** instead of multiple REST calls.

   Evolving APIs: Deprecate fields rather than versioning endpoints.

Django REST Framework

Django REST Framework

Back in 2005, when Django was first released, most websites consisted of one large **monolithic codebase**.

The back-end of database models, views, and URLs **were combined** with front-end templates to control the presentational layer of each web page.

But these days it is far more common for websites to adopt an API-first approach of formally **separating the back-end from the front-end**.

This allows a website to use a dedicated **JavaScript front-end framework**, such as **React**.

The major downside to an API-first approach is that it requires more configuration than a traditional Django application.

However **Django REST Framework** library removes much of that complexity.

It follows many of Django's traditional coding conventions.

Endpoints

A traditional website consists of web pages with HTML, CSS, images, JavaScript, and more.

There is a dedicated URL, such as **example.com/1/**, for each page.

A web API also relies on URLs and a corresponding one might be **example.com/api/1/**, but instead of serving up web pages consumable by humans **it produces API endpoints**.

Endpoint refers to a specific URL pattern that is mapped to a **View** or **Viewset** to handle HTTP requests and **provide a response**.

Endpoints define the available operations and resources exposed by an API.

They play a crucial role in defining the **structure** and **functionality** of the API.

- An API is a set of protocols and tools.
- An endpoint is a place on the API where exchange occurs. Endpoints are URIs (Uniform Resource Identifiers) on an API that an application can access.

Status Codes Response

Once your web browser has executed an HTTP Request on a URL, there is a way to understand the **status of the request**.

Thus there is a quite lengthy list of **HTTP Status Codes** available to accompany each HTTP response.

You can tell the general type of status code based on the following system:

- **2xx Success** - the action requested by the client was received, understood, and accepted
- **3xx Redirection** - the requested URL has moved
- **4xx Client Error** - there was an error, typically a bad URL request by the client
- **5xx Server Error** - the server failed to resolve a request

Installation settings

To install Django REST Framework simply run: **pip install djangorestframework**

Then we add in the settings.py of the Django project the REST framework to **INSTALLED_APPS** section.

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # 3rd party
    "rest_framework",  # new
    # Local
    "books.apps.BooksConfig",
]
```

Many professional Django developers will just include API logic in the related app while putting URLs under an **/api/** prefix.

To keep the API logic clear from the traditional Django logic, we will create a dedicated **apis** app for our project.

The web API will expose a **single endpoint**. To do this, we will need a new URL route, a new View, and a new **Serializer** file.

GenericAPIView

In traditional Django, Views are used to customize what data to send to the templates.

Django REST Framework views are similar except the end result is **serialized data in JSON format**, not the content for a web page.

To avoid confusion, some developers will define the API Views file **apiviews.py** or **api.py**.

Django REST Framework (**DRF**) provides a set of **generic Views and Mixins** that can help streamline the development of APIs by handling common CRUD operations.

GenericAPIView: It provides the core functionality for handling HTTP requests and responses, data serialization, and authentication and permission checks.

DestroyAPIView: provides a default implementation for handling DELETE requests to delete an object.

RetrieveAPIView: handles GET requests, retrieving a single object by its identifier.

CreateAPIView: provides a default implementation for handling POST requests to create a new object based on the provided data.

UpdateAPIView: It provides a default implementation for handling PUT and PATCH requests to update an object based on the provided data.

Example Book API

Set an **urlpattern** to access the API view

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("apis.urls")),
    path("", include("books.urls")),
]
```

The Book Model

```
# books/models.py
from django.db import models


class Book(models.Model):
    title = models.CharField(max_length=250)
    subtitle = models.CharField(max_length=250)
    author = models.CharField(max_length=100)
    isbn = models.CharField(max_length=13)

    def __str__(self):
        return self.title
```

The View for retrieve a list of books

```
# apis/views.py
from rest_framework import generics

from books.models import Book
from .serializers import BookSerializer


class BookAPIView(generics.ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

Serializers

Serializers allow complex data such as **querysets** and **model instances** to be converted to native Python datatypes.

Python datatypes can then be easily **rendered** into **JSON, XML** or other content types.

Serializers also provide **deserialization**, allowing parsed data to be converted back into complex types, after first validating the incoming data.

The serializers in REST framework work very similarly to Django's Form and ModelForm classes.

Serializer class provides a powerful and generic way to control the output of responses.

```
# apis/serializers.py
from rest_framework import serializers

from books.models import Book


class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ("title", "subtitle", "author", "isbn")
```

ModelSerializer class which provides a useful shortcut for creating serializers that deal with model instances and querysets.

Browsable API

Django REST Framework ships with a **built-in browsable API** that displays both the content and HTTP verbs associated with a given endpoint.

It displays the HTTP status code for the page, which is 200 meaning OK.

Specifies Content-Type is JSON.

Displays the information for every single book entry in a formatted manner.

Book Api

```
GET /api/
```

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

[
    {
        "title": "Django for Beginners",
        "subtitle": "Build Websites with Python and Django",
        "author": "William S. Vincent",
        "isbn": "978173546720"
    },
    {
        "title": "Django for APIs",
        "subtitle": "Build web APIs with Python and Django",
        "author": "William S. Vincent",
        "isbn": "978173546722"
    },
    {
        "title": "Django for Professionals",
        "subtitle": "Production websites with Python & Django",
        "author": "William S. Vincent",
        "isbn": "978173546723"
    }
]
```

CORS

Cross-Origin Resource Sharing (CORS) is a mechanism that allows resources (e.g., data, APIs) on a web page to be **requested from a different domain** than the one the page originated from.

It is a security feature implemented by web browsers to prevent malicious scripts from making **unauthorized cross-origin requests**.

When a web page makes an HTTP request to a different domain, the browser checks if the server hosting the requested resource allows cross-origin requests from the domain of the web page.

If the server permits the request, it includes **specific CORS headers in the response**, allowing the browser to proceed with the request.

It is necessary when the front-end will be on a **different local port** during development and a **completely different domain once deployed**.

pip install django-cors-headers

CORS details

Origin = **protocol + domain + port.**

http://localhost:3000 is a different origin from http://localhost:8000 (ports differ),

https://api.example.com is different from http://api.example.com (protocol differs).

Same-origin policy is the default: prevent malicious scripts on one site from quietly poking at another site's data.

Example: React/Vue/Angular dev server might run at http://localhost:3000 but API server might run at http://localhost:8000

Browser sends a request with an Origin header (**Origin: http://localhost:3000**) to the HTTP request.

Server checks the Origin and decides whether to allow it.

If the server is configured to allow http://localhost:3000, it responds with a header like: Access-Control-Allow-Origin: http://localhost:3000

If the server **does not include** this header (or explicitly forbids that origin), the browser will refuse to make the response available to your front-end JavaScript.

Read-write endpoint

If we want a read-write endpoint we can use particular **APIView**.

ListCreateAPIView, which is similar to the **ListAPIView** but allows for writes and therefore POST requests.

To make model available to be read, updated, or deleted; there is a built-in generic Django REST Framework View just for this purpose: **RetrieveUpdateDestroyAPIView**

models.py

```
class Post(models.Model):
    title = models.CharField(max_length=50)
    body = models.TextField()
    author = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.title
```

serializers.py

```
class PostSerializer(serializers.ModelSerializer):
    class Meta:
        fields = (
            "id",
            "author",
            "title",
            "body",
            "created_at",
        )
        model = Post
```

views.py

```
class PostList(generics.ListCreateAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer


class PostDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
```

127.0.0.1:8000/api/v1/1/ Guest

Django REST framework wsv

Post List / Post Detail

Post Detail

DELETE OPTIONS GET

RetrieveUpdateDestroyAPIView

```
PUT /api/v1/1/
```

```
HTTP 200 OK
Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "id": 1,
    "author": 1,
    "title": "1st Blog Post (edited)",
    "body": "Hello, World!",
    "created_at": "2022-02-07T17:34:00.892277Z"
}
```

Raw data HTML form

Author: wsv

Title: 1st Blog Post (edited)

Body: Hello, World!

PUT

PUT available

Permissions

Django REST Framework ships with several out-of-the-box permissions settings that we can use to secure our API.

These can be applied at a **project-level**, a **view-level,** or at any individual **model-level**.

Django REST Framework has a lot of configurations that are namespaced inside a single Django setting called **REST_FRAMEWORK**.

```
# django_project/settings.py
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.AllowAny",
    ],
}
```

There are actually four built-in project-level permissions:

- **AllowAny**: any user, authenticated or not, has full access
- **IsAuthenticated**: only authenticated, registered users have access
- **IsAdminUser**: only admins/superusers have access
- **IsAuthenticatedOrReadOnly**: unauthorized users can view, only authenticated users have write, edit, or delete privileges

View-Level Permissions

Permissions can be added at the view-level too for more granular control.

There are built-in various permission classes such as **IsAuthenticated**, **IsAdminUser**, **AllowAny**, **IsAuthenticatedOrReadOnly**, and more.

Additionally, you can create custom permission classes by subclassing **BasePermission** and implementing a new logic.

```
class PostDetail(generics.RetrieveUpdateDestroyAPIView):
    permission_classes = (permissions.IsAdminUser,)  # new
    queryset = Post.objects.all()
    serializer_class = PostSerializer
```

However if you navigate to see the Post Detail page an **HTTP 403 Forbidden** status code is displayed, if not an Admin.

Permissions are **evaluated in the order they are defined,** and the first permission class that denies access will trigger a permission denied response.

Custom Permissions

Django REST Framework relies on a **BasePermission** class from which all other permission classes inherit.

All the built-in permissions settings like **AllowAny** or **IsAuthenticated** simple extend BasePermission

```
class IsAuthorOrReadOnly(permissions.BasePermission):
    def has_permission(self, request, view):
        # Authenticated users only can see list view
        if request.user.is_authenticated:
            return True
        return False

    def has_object_permission(self, request, view, obj):
        # Read permissions are allowed to any request so we'll always
        # allow GET, HEAD, or OPTIONS requests
        if request.method in permissions.SAFE_METHODS:
            return True

        # Write permissions are only allowed to the author of a post
        return obj.author == request.user
```

For a custom permission class you can override one or both of these methods. **has_permission** works on list views while detail views execute both: first **has_permission** and then, if that passes, **has_object_permission**.

Basic Authentication

The most common form of HTTP authentication is known as **“Basic” Authentication**.

When a client makes an HTTP request, it is **forced** to send an approved authentication credential before access is granted.

The complete **request/response flow** looks like this:

1. Client makes an HTTP request.
2. Server responds with an HTTP response containing a **401 (Unauthorized)** status code and WWW-Authenticate HTTP header with details on how to authorize.
3. Client sends credentials back via the Authorization HTTP header.
4. Server checks credentials and responds with either 200 OK or 403 Forbidden status code.

Once approved, the client sends all **future requests with the Authorization HTTP header** credentials.

Session Authentication

The advantage of this “Basic” Authentication is its **simplicity**. But there are several major downsides:

1. First, on every single request the server must lookup and verify the username and password(**inefficient**).
2. Second, user credentials are being passed in clear text—not encrypted at all—over the internet (**insecure**)

Monolithic websites have long used an alternative authentication scheme that is a combination of **sessions and cookies.**

The client authenticates with its credentials and then receives a **session ID** from the server which is stored as a **cookie**.

This session ID is then **passed in the header** of every future HTTP request.

The server uses it to **look up a session** object containing information for a given user.

This approach is **stateful** because a record must be kept and maintained on both the server (the session object) and the client (the session ID).

Default Authentication

The default setting in Django REST Framework is actually a combination of Basic Authentication and Session Authentication.

Django's traditional session-based authentication system is used and the session ID is passed in the HTTP header on each request via Basic Authentication.

Advantage:

- more secure: user credentials are **only sent once**, not on every request/response cycle.
- more efficient: the server does not have to verify the user's credentials each time (matches the session ID to the session object)
- supports both browser-based use and simple script (curl/Postman) - or mobile-based clients without extra configuration.

Several downsides:

- session ID is only valid within the browser where login was performed (**not work across multiple domains**)
- session object must be kept up-to-date (**problem on multiple servers**)
- cookie is sent out for every single request (**inefficient**)

Token Authentication

Token-based authentication is **stateless**.

When client sends the initial user credentials to the server, a **unique token is generated**.

It is stored by the client as either a cookie or in local storage (file). The current best practice is to store tokens in a cookie with the **httpOnly and Secure cookie flags.**

This token is then passed in the header of each incoming HTTP request and the server uses it to verify that a user is authenticated.

The server itself **does not keep a record of the user**, just whether a **token is valid** or not.

Benefits:

- Since tokens are stored on the client, **scaling to multiple servers** is easy
- Tokens can be **shared** amongst multiple front-ends.

Downside:

- **Tokens size** can grow quite large (contains all user information) and it is sent on every request (manage its size).

Implementing token authentication

TokenAuthentication of Django REST Framework is quite basic.

It does not support setting tokens to expire. It generates one token per user.

```
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticated",
        ],
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.SessionAuthentication",
        "rest_framework.authentication.TokenAuthentication",  # new
    ],
}
```

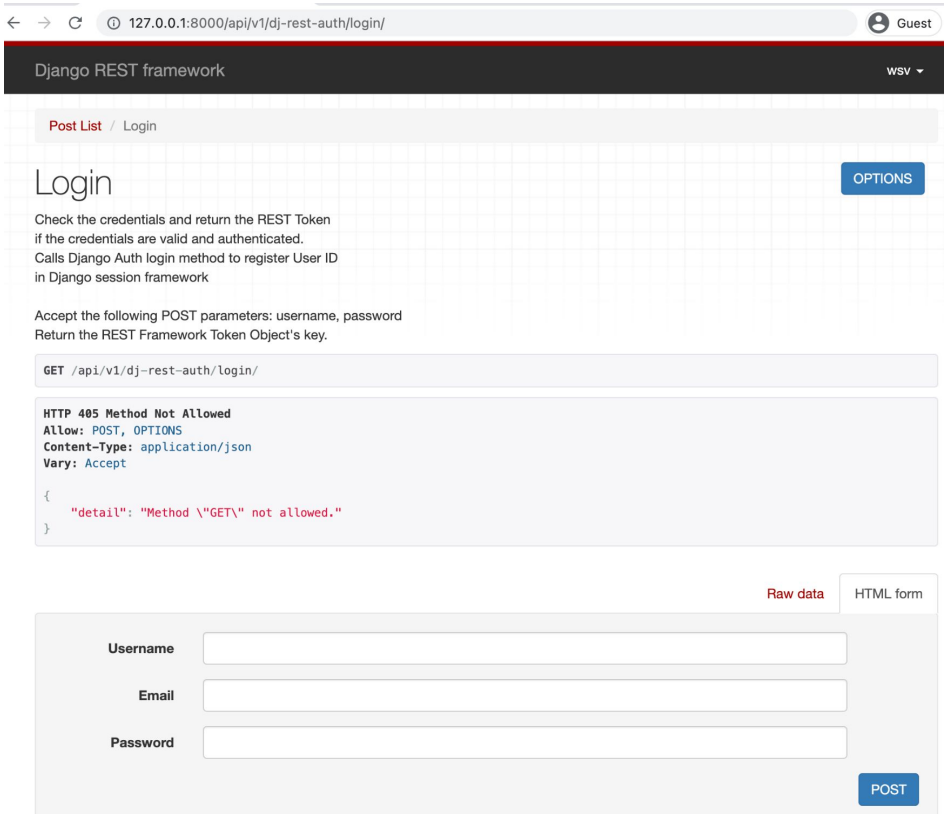
We also need to add the **authtoken** app which generates the tokens on the server in the **INSTALLED_APPS** setting.

dj-rest-auth

The login, logout, and password reset API endpoints come out-of-the-box with the popular **dj-rest-auth** package.

Update our django_project **urls.py** file with **dj_rest_auth urls**.

Set the URL routes to **api/v1/dj-rest-auth.** (URLs should have a dash not an underscore)



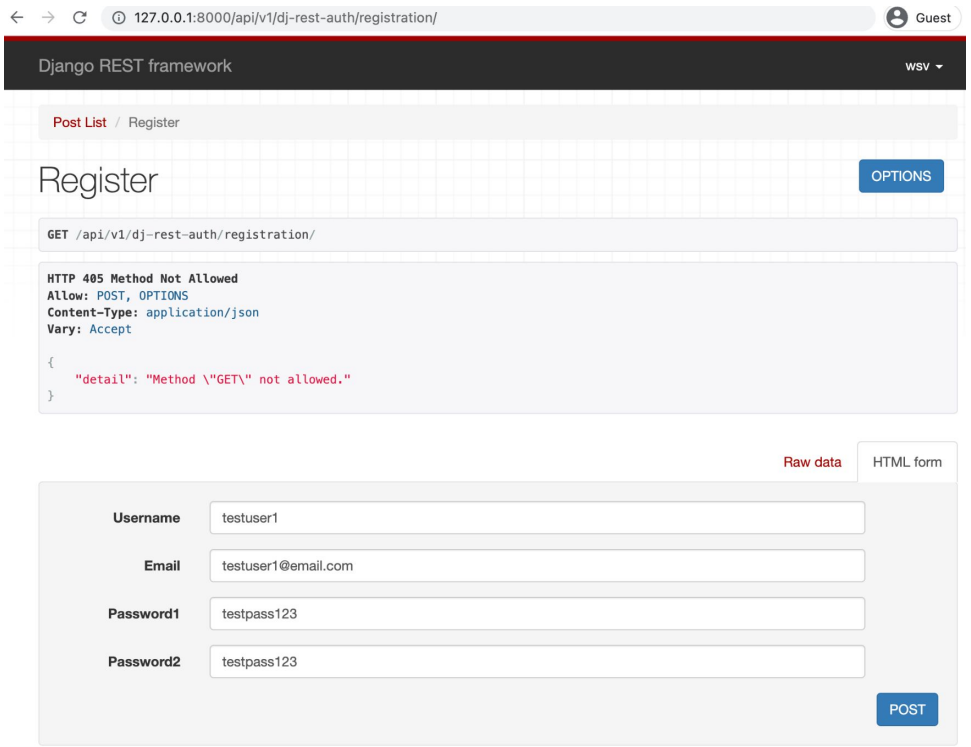
User Registration

A popular approach is to use the third-party package **django-allauth** which comes with user registration.

It has additional features to the Django auth system such as social authentication via Facebook, Google, Twitter, etc.

Add **dj_rest_auth.registration** from the **dj-rest-auth** package to have user registration endpoints.

A slight modification to the **settings.py** file is needed to make everything work.



Token generation

After clicking on the “POST” button the next screen shows the HTTP response from the server.

Our user registration POST was successful, hence the status code HTTP 201 Created at the top.

The return value key is the auth token for this new user.

```
POST /api/v1/dj-rest-auth/registration/
```

```
HTTP 201 Created
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "key": "d898c7581d08d97e1db294e7ecd45e0da13ee9e3"
}
```

JWT Authentication

JWT (**JSON Web Token**) is a compact, URL-safe method for representing pieces of information about the user between two parties. JWT Authentication is a way of securely transmitting information between the server and the client as a **token**.

It provides **stateless authentication**. The server does not need to store session information. The **client stores the token** and sends it with each request.

It helps in securely verifying the identity of the user through a signed token.

Suitable for APIs that need stateless authentication, especially in mobile apps, web apps, or microservices.

On successful login, the server creates a JWT token that includes the user's identity and other claims (like expiration time).

The client stores this token (often in localStorage or sessionStorage for web apps).

For every subsequent request, the client sends this token in the HTTP headers (typically as **Authorization: Bearer <token>**).

The server verifies the token, and if valid, grants access to the requested resources.

django-filter

A library that simplifies adding filter functionality to querysets in views and APIs.

Lets users filter data (e.g., movies, books, users) by defining **a set of filter fields** instead of writing query logic.

Integrates seamlessly with Django's generic views and Django REST Framework.

lookup_expr='icontains': matches substrings regardless of case.

field_name='year' + **lookup_expr='gt'** or **'lt'**: creates "greater than" and "less than" filters for the same year field.

creator__username: follows Django's lookup syntax to filter by a related model's field.

```
class MovieFilter(filters.FilterSet):
    # Filters title using case-insensitive "contains"
    title = filters.CharFilter(lookup_expr='icontains')
    # Filters genre using case-insensitive "contains"
    genre = filters.CharFilter(lookup_expr='icontains')
    # Exact match on year
    year = filters.NumberFilter()
    # Greater-than filter on year
    year__gt = filters.NumberFilter(
        field_name='year', lookup_expr='gt'
    )
    # Less-than filter on year
    year__lt = filters.NumberFilter(
        field_name='year', lookup_expr='lt'
    )
    # Filter by related user's username (creator.username)
    creator__username = filters.CharFilter(lookup_expr='icontains')

    class Meta:
        model = Movie
        # Expose these fields to be used in query parameters
        fields = [
            'title',
            'genre',
            'year',
            'year__gt',
            'year__lt',
            'creator__username'
        ]
```

Pagination

Breaks large **querysets** into smaller "pages" of results to reduce payload size and improve performance.

Built-in Classes: DRF provides several pagination styles (e.g., **PageNumberPagination**, **LimitOffsetPagination**, **CursorPagination**).

Easier to navigate large datasets (e.g., "Next" / "Previous" buttons in a **frontend**).

Flexibility: Clients control page size when allowed via page_size_query_param.

```python
from rest_framework.pagination import PageNumberPagination

class CustomPagination(PageNumberPagination):
    # Default items per page
    page_size = 10
    # Allow client to set ?page_size= value
    page_size_query_param = 'page_size'
```

```python
from rest_framework import viewsets
from .models import Item
from .serializers import ItemSerializer

class ItemViewSet(viewsets.ModelViewSet):
    queryset = Item.objects.all()
    serializer_class = ItemSerializer
    pagination_class = CustomPagination
```

```json
{
  "count": 123,
  "next": "http://.../api/items/?page=2",
  "previous": null,
  "results": [ /* first 10 items */ ]
}
```